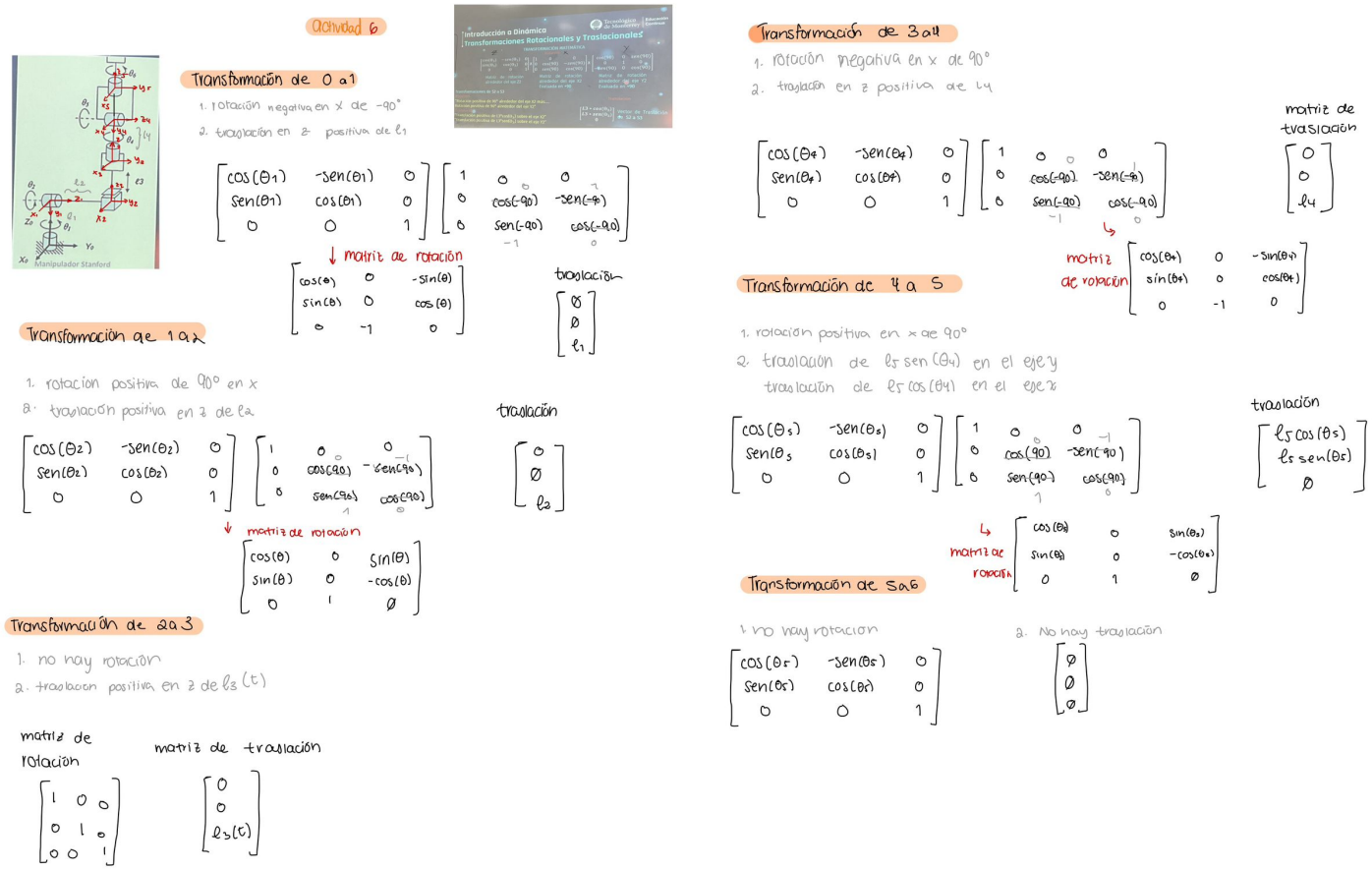
Actividad 6

Ximena Ortiz Gómez A01735100

Análisis




```
%%%%%
%Limpieza de pantalla
clear all
close all
clc

tic
%Declaración de variables simbólicas
syms th1(t) th2(t) th4(t) th5(t) th6(t)
syms th1p(t) th2p(t) th4p(t) th5p(t) th6p(t)   %Velocidades de cada articulación
syms l1 l2 l3(t) l4 l5 l6
syms l1p(t) l2p(t) l3p(t)   %Velocidades de cada articulación
syms th1pp(t) th2pp(t) th4pp(t) th5pp(t) th6pp(t) %Aceleraciones de cada
articulación
syms l3pp(t) %Aceleraciones de cada articulación
syms m1 m2 m3 m4 m5 m6 Ixx1 Iyy1 Izz1 Ixx2 Iyy2 Izz2 Ixx3 Iyy3 Izz3 Ixx4 Iyy4 Izz4
Ixx5 Iyy5 Izz5 Ixx6 Iyy6 Izz6 %Masas y matrices de Inercia
```

```
syms lc1 lc2 lc3 lc4 lc5 lc6 %l=longitud de eslabones y lc=distancia al centro de
masa de cada eslabón
syms pi g a cero

 %Creamos el vector de coordenadas articulares
  Q= [th1; th2; l3; th4; th5; th6];
 %disp('Coordenadas generalizadas');
 %pretty (Q);

 %Creamos el vector de velocidades articulares
 Qp= [th1p; th2p; l3p; th4p; th5p; th6p];
 %disp('Velocidades generalizadas');
 %pretty (Qp);
 %Creamos el vector de aceleraciones articulares
 Qpp= [th1pp; th2pp; l3pp; th4pp; th5pp; th6pp];
 %disp('Aceleraciones generalizadas');
 %pretty (Qpp);

%Configuración del robot, 0 para junta rotacional, 1 para junta prismática
RP=[0 0 1 0 0 0];

%Número de grado de libertad del robot
GDL= size(RP,2);
GDL_str= num2str(GDL);

%Articulación 1
%Posición de la articulación 1 respecto a 0
P(:,:,1)= [0; 0; l1];
%Matriz de rotación de la junta 1 respecto a 0....
R(:,:,1)= [cos(th1)  0  -sin(th1);
           sin(th1)  0  cos(th1);
           0        -1       0];

%Articulación 2
%Posición de la articulación 2 respecto a 1
P(:,:,2)= [0; 0; l2];
%Matriz de rotación de la junta 1 respecto a 0
R(:,:,2)= [cos(th2)  0  sin(th2);
           sin(th2)  0  -cos(th2);
           0        1       0];

%Articulación 3
%Posición de la articulación 2 respecto a 1
P(:,:,3)= [0; 0; l3];
%Matriz de rotación de la junta 1 respecto a 0
R(:,:,3)= [1  0  0;
           0  1  0;
           0  0  1];

%Articulación 4
```

```
%Posición de la articulación 2 respecto a 1
P(:,:,4)= [0; 0; l4];
%Matriz de rotación de la junta 1 respecto a 0
R(:,:,4)= [cos(th4)  0  -sin(th4);
           sin(th4)  0   cos(th4);
           0        -1       0];

%Articulación 5
%Posición de la articulación 2 respecto a 1
P(:,:,5)= [l5*cos(th5); l5*sin(th5); 0];
%Matriz de rotación de la junta 1 respecto a 0
R(:,:,5)= [cos(th5)  0    sin(th5);
           sin(th5)  0   -cos(th5);
           0         1       0];

%Articulación 6
%Posición de la articulación 2 respecto a 1
P(:,:,6)= [0; 0; 0];
%Matriz de rotación de la junta 1 respecto a 0 %%%%%%%%%%%%%%%%%%%%%%
R(:,:,6)= [cos(th5)  -sin(th5)  0 ;
           sin(th5)  cos(th5)   0;
           0         0          1];



%Creamos un vector de ceros
Vector_Zeros= zeros(1, 3);

%Inicializamos las matrices de transformación Homogénea locales
A(:,:,GDL)=simplify([R(:,:,GDL) P(:,:,GDL); Vector_Zeros 1]);
%Inicializamos las matrices de transformación Homogénea globales
T(:,:,GDL)=simplify([R(:,:,GDL) P(:,:,GDL); Vector_Zeros 1]);
%Inicializamos las posiciones vistas desde el marco de referencia inercial
PO(:,:,GDL)= P(:,:,GDL);
%Inicializamos las matrices de rotación vistas desde el marco de referencia inercial
RO(:,:,GDL)= R(:,:,GDL);


for i = 1:GDL
    i_str= num2str(i);
   %disp(strcat('Matriz de Transformación local A', i_str));
    A(:,:,i)=simplify([R(:,:,i) P(:,:,i); Vector_Zeros 1]);
   %pretty (A(:,:,i));

   %Globales
    try
       T(:,:,i)= T(:,:,i-1)*A(:,:,i);
    catch
       T(:,:,i)= A(:,:,i);
    end
```

```
%     disp(strcat('Matriz de Transformación global T', i_str));
    T(:,:,i)= simplify(T(:,:,i));
%     pretty(T(:,:,i))

    RO(:,:,i)= T(1:3,1:3,i);
    PO(:,:,i)= T(1:3,4,i);
    %pretty(RO(:,:,i));
    %pretty(PO(:,:,i));
end


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%CALCULAMOS LAS VELOCIDADES PARA CADA ESLABÓN%%%%%%%
%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%  VELOCIDADES PARA ESLABÓN 6  %%%%%%%%%%

%Calculamos el jacobiano lineal de forma analítica
Jv_a6(:,GDL)=PO(:,:,GDL);
Jw_a6(:,GDL)=PO(:,:,GDL);

for k= 1:GDL
    if RP(k)==0
       %Para las juntas de revolución
        try
            Jv_a6(:,k)= cross(RO(:,3,k-1), PO(:,:,GDL)-PO(:,:,k-1));
            Jw_a6(:,k)= RO(:,3,k-1);
        catch
            Jv_a6(:,k)= cross([0,0,1], PO(:,:,GDL));%Matriz de rotación de 0 con
respecto a 0 es la Matriz Identidad, la posición previa tambien será 0
            Jw_a6(:,k)=[0,0,1];%Si no hay matriz de rotación previa se obtiene la
Matriz identidad
         end
     else
%         %Para las juntas prismáticas
        try
            Jv_a6(:,k)= RO(:,3,k-1);
        catch
            Jv_a6(:,k)=[0,0,1];
        end
            Jw_a6(:,k)=[0,0,0];
     end
 end

%Obtenemos SubMatrices de Jacobianos
Jv_a6= simplify (Jv_a6);
Jw_a6= simplify (Jw_a6);
%disp('Jacobiano lineal obtenido de forma analítica');
%pretty (Jv_a);
%disp('Jacobiano ángular obtenido de forma analítica');
%pretty (Jw_a);
```

```
%Matriz de Jacobiano Completa
%disp('Matriz de Jacobiano');
Jac6= [Jv_a6;
      Jw_a6];
Jacobiano6= simplify(Jac6);
% pretty(Jacobiano);

%Obtenemos vectores de Velocidades Lineales y Angulares para el eslabón 6
disp('Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 6');
```

```
Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 6
```

```
V6=simplify (Jv_a6*Qp);
pretty(V6)
```

```
/ cos(th1(t)) th2p(t) #1 - th1p(t) (l2 cos(th1(t)) + l4 sin(th1(t)) sin(th2(t)) + l5 cos(th5(t)) (cos(th1(t)) sin(th
|
| sin(th1(t)) th2p(t) #1 - th1p(t) (l2 sin(th1(t)) - l4 cos(th1(t)) sin(th2(t)) + l5 cos(th5(t)) (sin(th1(t)) sin(th
|
\

where

   #1 == l4 cos(th2(t)) + cos(th2(t)) l3(t) - l5 cos(th2(t)) sin(th5(t)) - l5 cos(th4(t)) cos(th5(t)) sin(th2(t))
```

```
 disp('Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 6');
```

```
Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 6
```

```
W6=simplify (Jw_a6*Qp);
pretty(W6)
```

```
/ cos(th1(t)) sin(th2(t)) th4p(t) - th6p(t) (sin(th5(t)) (sin(th1(t)) sin(th4(t)) - cos(th1(t)) cos(th2(t)) cos(th4
|
| th5p(t) (cos(th1(t)) cos(th4(t)) - cos(th2(t)) sin(th1(t)) sin(th4(t))) + th6p(t) (sin(th5(t)) (cos(th1(t)) sin(th
|
\                                                          th1p(t) + th6p(t) (cos(th2(t)) cos(th5(t)) - cos(th4(t)
```

```
%%%%%%%%%%  VELOCIDADES PARA ESLABÓN 5  %%%%%%%%%%

%Calculamos el jacobiano lineal y angular de forma analítica
Jv_a5(:,GDL-1)=PO(:,:,GDL-1);
Jw_a5(:,GDL-1)=PO(:,:,GDL-1);

for k= 1:GDL-1
    if RP(k)==0
       %Para las juntas de revolución
        try
            Jv_a5(:,k)= cross(RO(:,3,k-1), PO(:,:,GDL-1)-PO(:,:,k-1));
            Jw_a5(:,k)= RO(:,3,k-1);
        catch
```

```
            Jv_a5(:,k)= cross([0,0,1], PO(:,:,GDL-1));%Matriz de rotación de 0 con
respecto a 0 es la Matriz Identidad, la posición previa tambien será 0
            Jw_a5(:,k)=[0,0,1];%Si no hay matriz de rotación previa se obtiene la
Matriz identidad
         end
     else
%         %Para las juntas prismáticas
        try
            Jv_a5(:,k)= RO(:,3,k-1);
        catch
            Jv_a5(:,k)=[0,0,1];
        end
            Jw_a5(:,k)=[0,0,0];
     end
 end

%Obtenemos SubMatrices de Jacobianos
Jv_a5= simplify (Jv_a5);
Jw_a5= simplify (Jw_a5);
%disp('Jacobiano lineal obtenido de forma analítica');
%pretty (Jv_a);
%disp('Jacobiano ángular obtenido de forma analítica');
%pretty (Jw_a);

%Matriz de Jacobiano Completa
%disp('Matriz de Jacobiano');
Jac5= [Jv_a5;
      Jw_a5];
Jacobiano5= simplify(Jac5);
% pretty(Jacobiano);

%Obtenemos vectores de Velocidades Lineales y Angulares para el eslabón 5
disp('Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 5');
```

```
Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 5
```

```
Qp=Qp(t);
V5=simplify (Jv_a5*Qp(1:5));
pretty(V5)
```

```
/ cos(th1(t)) th2p(t) #1 - th1p(t) (l2 cos(th1(t)) + l4 sin(th1(t)) sin(th2(t)) + l5 cos(th5(t)) (cos(th1(t)) sin(th
|
| sin(th1(t)) th2p(t) #1 - th1p(t) (l2 sin(th1(t)) - l4 cos(th1(t)) sin(th2(t)) + l5 cos(th5(t)) (sin(th1(t)) sin(th
|
\

where

   #1 == l4 cos(th2(t)) + cos(th2(t)) l3(t) - l5 cos(th2(t)) sin(th5(t)) - l5 cos(th4(t)) cos(th5(t)) sin(th2(t))
```

```
 disp('Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 2');
```

```
Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 2
```

```
W5=simplify (Jw_a5*Qp(1:5));
pretty(W5)
```

```
/ cos(th1(t)) sin(th2(t)) th4p(t) - sin(th1(t)) th2p(t) - th5p(t) (cos(th4(t)) sin(th1(t)) + cos(th1(t)) cos(th2(t)
|
| th5p(t) (cos(th1(t)) cos(th4(t)) - cos(th2(t)) sin(th1(t)) sin(th4(t))) + cos(th1(t)) th2p(t) + sin(th1(t)) sin(th
|
\                                th1p(t) + cos(th2(t)) th4p(t) + sin(th2(t)) sin(th4(t)) th5p(t)
```

```
%%%%%%%%%%  VELOCIDADES PARA ESLABÓN 4  %%%%%%%%%%%%%%%%%%%%%%%%%%

%Calculamos el jacobiano lineal y angular de forma analítica
Jv_a4(:,GDL-2)=PO(:,:,GDL-2);
Jw_a4(:,GDL-2)=PO(:,:,GDL-2);

for k= 1:GDL-2
    if RP(k)==0
       %Para las juntas de revolución
        try
            Jv_a4(:,k)= cross(RO(:,3,k-1), PO(:,:,GDL-2)-PO(:,:,k-1));
            Jw_a4(:,k)= RO(:,3,k-1);
        catch
            Jv_a4(:,k)= cross([0,0,1], PO(:,:,GDL-2));%Matriz de rotación de 0 con
respecto a 0 es la Matriz Identidad, la posición previa tambien será 0
            Jw_a4(:,k)=[0,0,1];%Si no hay matriz de rotación previa se obtiene la
Matriz identidad
         end
     else
%         %Para las juntas prismáticas
        try
            Jv_a4(:,k)= RO(:,3,k-1);
        catch
            Jv_a4(:,k)=[0,0,1];
        end
            Jw_a4(:,k)=[0,0,0];
     end
 end

%Obtenemos SubMatrices de Jacobianos
Jv_a4= simplify (Jv_a4);
Jw_a4= simplify (Jw_a4);
%disp('Jacobiano lineal obtenido de forma analítica');
%pretty (Jv_a);
%disp('Jacobiano ángular obtenido de forma analítica');
%pretty (Jw_a);

%Matriz de Jacobiano Completa
%disp('Matriz de Jacobiano');
```

```
Jac4= [Jv_a4;
      Jw_a4];
Jacobiano4= simplify(Jac4);
% pretty(Jacobiano);

%Obtenemos vectores de Velocidades Lineales y Angulares para el eslabón 1
disp('Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 4');
```

```
Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 4
```

```
V4=simplify (Jv_a4*Qp(1:4));
pretty(V4)
```

```
/ cos(th1(t)) sin(th2(t)) l3p(t) - th1p(t) (l2 cos(th1(t)) + l4 sin(th1(t)) sin(th2(t)) + sin(th1(t)) sin(th2(t)) l3
|
| th1p(t) (l4 cos(th1(t)) sin(th2(t)) - l2 sin(th1(t)) + cos(th1(t)) sin(th2(t)) l3(t)) + sin(th1(t)) sin(th2(t)) l3
|
\                                                        cos(th2(t)) l3p(t) - sin(th2(t)) th2p(t) (l4 + l3(t))
```

```
disp('Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 4');
```

```
Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 4
```

```
W4=simplify (Jw_a4*Qp(1:4));
pretty(W4)
```

```
/ cos(th1(t)) sin(th2(t)) th4p(t) - sin(th1(t)) th2p(t) \
|                                                       |
| cos(th1(t)) th2p(t) + sin(th1(t)) sin(th2(t)) th4p(t) |
|                                                       |
\            th1p(t) + cos(th2(t)) th4p(t)             /
```

```
%%%%%%%%%%  VELOCIDADES PARA ESLABÓN 3  %%%%%%%%%%%%%%%%%%%%%%%%%%

%Calculamos el jacobiano lineal y angular de forma analítica
Jv_a3(:,GDL-3)=PO(:,:,GDL-3);
Jw_a3(:,GDL-3)=PO(:,:,GDL-3);

for k= 1:GDL-3
    if RP(k)==0
       %Para las juntas de revolución
        try
            Jv_a3(:,k)= cross(RO(:,3,k-1), PO(:,:,GDL-3)-PO(:,:,k-1));
            Jw_a3(:,k)= RO(:,3,k-1);
        catch
            Jv_a3(:,k)= cross([0,0,1], PO(:,:,GDL-3));%Matriz de rotación de 0 con
respecto a 0 es la Matriz Identidad, la posición previa tambien será 0
            Jw_a3(:,k)=[0,0,1];%Si no hay matriz de rotación previa se obtiene la
Matriz identidad
         end
     else
%         %Para las juntas prismáticas
```

```
        try
            Jv_a3(:,k)= RO(:,3,k-1);
        catch
            Jv_a3(:,k)=[0,0,1];
        end
            Jw_a3(:,k)=[0,0,0];
    end
 end

%Obtenemos SubMatrices de Jacobianos
Jv_a3= simplify (Jv_a3);
Jw_a3= simplify (Jw_a3);
%disp('Jacobiano lineal obtenido de forma analítica');
%pretty (Jv_a);
%disp('Jacobiano ángular obtenido de forma analítica');
%pretty (Jw_a);

%Matriz de Jacobiano Completa
%disp('Matriz de Jacobiano');
Jac3= [Jv_a3;
      Jw_a3];
Jacobiano3= simplify(Jac3);
% pretty(Jacobiano);

%Obtenemos vectores de Velocidades Lineales y Angulares para el eslabón 1
disp('Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 3');
```

```
Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 3
```

```
V3=simplify (Jv_a3*Qp(1:3));
pretty(V3)
```

```
/ cos(th1(t)) sin(th2(t)) l3p(t) - th1p(t) (l2 cos(th1(t)) + sin(th1(t)) sin(th2(t)) l3(t)) + cos(th1(t)) cos(th2(t
|
| sin(th1(t)) sin(th2(t)) l3p(t) - th1p(t) (l2 sin(th1(t)) - cos(th1(t)) sin(th2(t)) l3(t)) + cos(th2(t)) sin(th1(t
|
\                                          cos(th2(t)) l3p(t) - sin(th2(t)) l3(t) th2p(t)
```

```
disp('Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 3');
```

```
Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 3
```

```
W3=simplify (Jw_a3*Qp(1:3));
pretty(W3)
```

```
/ -sin(th1(t)) th2p(t) \
|                      |
|  cos(th1(t)) th2p(t) |
|                      |
\        th1p(t)       /
```

```
%%%%%%%%%%  VELOCIDADES PARA ESLABÓN 2  %%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%Calculamos el jacobiano lineal y angular de forma analítica
Jv_a2(:,GDL-4)=PO(:,:,GDL-4);
Jw_a2(:,GDL-4)=PO(:,:,GDL-4);

for k= 1:GDL-4
    if RP(k)==0
       %Para las juntas de revolución
        try
            Jv_a2(:,k)= cross(RO(:,3,k-1), PO(:,:,GDL-4)-PO(:,:,k-1));
            Jw_a2(:,k)= RO(:,3,k-1);
        catch
            Jv_a2(:,k)= cross([0,0,1], PO(:,:,GDL-4));%Matriz de rotación de 0 con
respecto a 0 es la Matriz Identidad, la posición previa tambien será 0
            Jw_a2(:,k)=[0,0,1];%Si no hay matriz de rotación previa se obtiene la
Matriz identidad
         end
     else
%         %Para las juntas prismáticas
        try
            Jv_a2(:,k)= RO(:,3,k-1);
        catch
            Jv_a2(:,k)=[0,0,1];
        end
            Jw_a2(:,k)=[0,0,0];
     end
 end

%Obtenemos SubMatrices de Jacobianos
Jv_a2= simplify (Jv_a2);
Jw_a2= simplify (Jw_a2);
%disp('Jacobiano lineal obtenido de forma analítica');
%pretty (Jv_a);
%disp('Jacobiano ángular obtenido de forma analítica');
%pretty (Jw_a);

%Matriz de Jacobiano Completa
%disp('Matriz de Jacobiano');
Jac2= [Jv_a2;
      Jw_a2];
Jacobiano2= simplify(Jac2);
% pretty(Jacobiano);

%Obtenemos vectores de Velocidades Lineales y Angulares para el eslabón 2
disp('Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 2');
```

```
Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 2
```

```
V2=simplify (Jv_a2*Qp(1:2));
pretty(V2)
```

```
/ -l2 cos(th1(t)) th1p(t) \
|                         |
| -l2 sin(th1(t)) th1p(t) |
|                         |
\           0             /
```

```
disp('Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 2');
```

```
Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 2
```

```
W2=simplify (Jw_a2*Qp(1:2));
pretty(W2)
```

```
/ -sin(th1(t)) th2p(t) \
|                      |
|  cos(th1(t)) th2p(t) |
|                      |
\        th1p(t)       /
```

```
%%%%%%%%%%  VELOCIDADES PARA ESLABÓN 1 %%%%%%%%%%%%%%%%%%%%%%%%%%

%Calculamos el jacobiano lineal y angular de forma analítica
Jv_a1(:,GDL-5)=PO(:,:,GDL-5);
Jw_a1(:,GDL-5)=PO(:,:,GDL-5);

for k= 1:GDL-5
    if RP(k)==0
       %Para las juntas de revolución
        try
            Jv_a1(:,k)= cross(RO(:,3,k-1), PO(:,:,GDL-5)-PO(:,:,k-1));
            Jw_a1(:,k)= RO(:,3,k-1);
        catch
            Jv_a1(:,k)= cross([0,0,1], PO(:,:,GDL-5));%Matriz de rotación de 0 con
respecto a 0 es la Matriz Identidad, la posición previa tambien será 0
            Jw_a1(:,k)=[0,0,1];%Si no hay matriz de rotación previa se obtiene la
Matriz identidad
         end
     else
%         %Para las juntas prismáticas
        try
            Jv_a1(:,k)= RO(:,3,k-1);
        catch
            Jv_a1(:,k)=[0,0,1];
        end
            Jw_a1(:,k)=[0,0,0];
     end
 end

%Obtenemos SubMatrices de Jacobianos
Jv_a1= simplify (Jv_a1);
Jw_a1= simplify (Jw_a1);
```

```
%disp('Jacobiano lineal obtenido de forma analítica');
%pretty (Jv_a);
%disp('Jacobiano ángular obtenido de forma analítica');
%pretty (Jw_a);

%Matriz de Jacobiano Completa
%disp('Matriz de Jacobiano');
Jac1= [Jv_a1;
      Jw_a1];
Jacobiano1= simplify(Jac1);
% pretty(Jacobiano);

%Obtenemos vectores de Velocidades Lineales y Angulares para el eslabón 1
disp('Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 1');
```

```
Velocidad lineal obtenida mediante el Jacobiano lineal del Eslabón 1
```

```
V1=simplify (Jv_a1*Qp(1:1));
pretty(V1)
```

```
/ 0 \
|   |
| 0 |
|   |
\ 0 /
```

```
disp('Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 1');
```

```
Velocidad angular obtenida mediante el Jacobiano angular del Eslabón 1
```

```
W1=simplify (Jw_a1*Qp(1:1));
pretty(W1)
```

```
/    0    \
|         |
|    0    |
|         |
\ th1p(t) /
```

```
%%%
```

```
I1=[Ixx1 0 0;
    0 Iyy1 0;
    0 0 Izz1];

I2=[Ixx2 0 0;
    0 Iyy2 0;
    0 0 Izz2];

I3=[Ixx3 0 0;
    0 Iyy3 0;
    0 0 Izz3];

I4=[Ixx4 0 0;
    0 Iyy4 0;
    0 0 Izz4];

I5=[Ixx5 0 0;
    0 Iyy5 0;
    0 0 Izz5];

I6=[Ixx6 0 0;
    0 Iyy6 0;
    0 0 Izz6];

%Función de energía cinética

%Calculamos la energía cinética para cada uno de los eslabones%%%%%%%%%

%Eslabón 1
V1_Total= V1+cross(W1,P01);
K1= (1/2*m1*(V1_Total))'*((V1_Total)) + (1/2*W1)'*(I1*W1);
disp('Energía Cinética en el Eslabón 1');
```

```
Energía Cinética en el Eslabón 1
```

```
K1= simplify (K1);
pretty (K1);
```

```
                2
Izz1 |th1p(t)|
---------------
       2
```

```
%Eslabón 2
V2_Total= V2+cross(W2,P12);
K2= (1/2*m2*(V2_Total))'*((V2_Total)) + (1/2*W2)'*(I2*W2);
disp('Energía Cinética en el Eslabón 2');
```

```
Energía Cinética en el Eslabón 2
```

```
K2= simplify (K2);
```

```
pretty (K2);
```

```
Izz2 #7   Iyy2 #6 cos(th1(t)) #4   Ixx2 #6 sin(th1(t)) #3   cos(th1(t)) #4 m2 #5 #1   sin(th1(t)) #3 m2 #5 #1
------- + ---------------------- + ---------------------- - ------------------------ - ------------------------
   2                 2                        2                        #2                        #2

where

                      lc2 th2p(t)
   #1 == l2 th1p(t) - -----------
                            2

   #2 == 4 l2 lc2 th1p(t) th2p(t)


   #3 == sin(th1(t))


   #4 == cos(th1(t))

                   2                        2
   #5 == l2 #6 |lc2|  th1p(t) - 2 lc2 #7 |l2|  th2p(t)

                 2
   #6 == |th2p(t)|

                 2
   #7 == |th1p(t)|
```

```
%Eslabón 3
V3_Total= V3+cross(W3,P23);
K3= (1/2*m3*(V3_Total))'*((V3_Total)) + (1/2*W3)'*(I3*W3);
disp('Energía Cinética en el Eslabón 3');
```

```
Energía Cinética en el Eslabón 3
```

```
K3= simplify (K3);
pretty (K3);
```

```
   /                                                   _______   ___                          \
__ | ______          _______      __   _____           th2p(t) #1 lc3   _____ _______         | / lc3 sin(th1(t)) th2p(t
m3 | l3p(t) #1 #3 - th1p(t) (#1 l2 - l3(t) #2 #3) + -------------- + l3(t) th2p(t) #4 #1 | | ----------------------
   \                                                       2                               / \            2
-----------------------------------------------------------------------------------------------------------------------


where


   #1 == sin(th1(t))


   #2 == cos(th1(t))


   #3 == sin(th2(t))


   #4 == cos(th2(t))
```

```
%Eslabón 4
V4_Total= V4+cross(W4,P34);
K4= (1/2*m4*(V4_Total))'*((V4_Total)) + (1/2*W4)'*(I3*W4);
disp('Energía Cinética en el Eslabón 4');
```

```
Energía Cinética en el Eslabón 4
```

```
K4= simplify (K4);
pretty (K4);
```

```
                                                  / ___
                                              __  | lc4 (#7 + #6)   _______        __   _____                                  __     _____
                                              m4  | ------------- - th1p(t) (#10 l2 + l3(t) #11 #9 + #11 #9 l4) + l3p(t
     / #8   #5 \           / #7   #6 \            \       2
Ixx3 | -- - -- | #1 + Iyy3 | -- + -- | #2 + ----------------------------------------------------------------------------
     \  2    2 /           \  2    2 /

where

   #1 == sin(th1(t)) th2p(t) - cos(th1(t)) sin(th2(t)) th4p(t)

   #2 == cos(th1(t)) th2p(t) + sin(th1(t)) sin(th2(t)) th4p(t)

   #3 == _____ + __
          l3(t)  l4

   #4 == cos(_______)
             th2(t))

   #5 == _______ #10 #9
          th4p(t)

   #6 == _______ #11 #9
          th4p(t)

   #7 == _______ #10
          th2p(t)

   #8 == _______ #11
          th2p(t)

   #9 == sin(_______)
             th2(t))

   #10 == cos(_______)
              th1(t))

   #11 == sin(_______)
              th1(t))
```

```
%Eslabón 5
V5_Total= V3+cross(W5,P45);
K5= (1/2*m5*(V5_Total))'*((V5_Total)) + (1/2*W5)'*(I5*W5);
disp('Energía Cinética en el Eslabón 5');
```

```
Energía Cinética en el Eslabón 5
```

```
K5= simplify (K5);
pretty (K5);
```

```
                                  __ / lc5 cos(th5(t)) #2                        lc5 sin(th5(t)) #1
     / _______               \      m5 | ------------------ - cos(th2(t)) l3p(t) + ------------------ + sin(th2(t)) l3
     | th1p(t)   #15   #14 |         \          2                                          2
Izz5 | ------- + --- + --- | #3 + ----------------------------------------------------------------------------------
     \    2       2     2  /

where

   #1 == th5p(t) (cos(th4(t)) sin(th1(t)) + cos(th1(t)) cos(th2(t)) sin(th4(t))) + sin(th1(t)) th2p(t) - cos(th1(t))

   #2 == th5p(t) (cos(th1(t)) cos(th4(t)) - cos(th2(t)) sin(th1(t)) sin(th4(t))) + cos(th1(t)) th2p(t) + sin(th1(t))

   #3 == th1p(t) + cos(th2(t)) th4p(t) + sin(th2(t)) sin(th4(t)) th5p(t)

   #4 == sin(_______)
                th5(t)

   #5 == cos(_______)
                th5(t)

   #6 == _______ (#16 #13 - #20 #17 #19)
         th5p(t)

   #7 == _______ (#13 #17 + #16 #20 #19)
         th5p(t)

   #8 == _______ + #15 + #14
         th1p(t)

   #9 == _______ #16 #18
         th4p(t)

   #10 == _______ #17 #18
          th4p(t)

   #11 == _______ #16
          th2p(t)

   #12 == _______ #17
          th2p(t)

   #13 == cos(_______)
              th4(t)

   #14 == _______ #18 #19
          th5p(t)

   #15 == _______ #20
          th4p(t)

   #16 == cos(_______)
              th1(t)

   #17 == sin(_______)
              th1(t)

   #18 == sin(_______)
              th2(t)

   #19 == sin(_______)
              th4(t)
```

```
   #20 == cos(th2(t))
```

```
%Eslabón 6
V6_Total= V6+cross(W6,P56);
K6= (1/2*m6*(V6_Total))'*((V6_Total)) + (1/2*W6)'*(I6*W6);
disp('Energía Cinética en el Eslabón 6');
```

```
Energía Cinética en el Eslabón 6
```

```
K6= simplify (K6);
pretty (K6);
```

```
                                                                                                                  / _______   _______          _____
                                                                                                                  | th1p(t)   th6p(t) #12   th4p(t
Izz6 (th1p(t) + th6p(t) #2 + cos(th2(t)) th4p(t) + sin(th2(t)) sin(th4(t)) th5p(t)) | ------- + ----------- + ------
                                                                                                                  \    2           2             2

where

   #1 == l4 cos(th2(t)) + cos(th2(t)) l3(t) - l5 cos(th2(t)) sin(th5(t)) - l5 cos(th4(t)) cos(th5(t)) sin(th2(t))

   #2 == cos(th2(t)) cos(th5(t)) - cos(th4(t)) sin(th2(t)) sin(th5(t))

   #3 == cos(th1(t)) cos(th4(t)) - cos(th2(t)) sin(th1(t)) sin(th4(t))

   #4 == sin(th1(t)) sin(th4(t)) - cos(th1(t)) cos(th2(t)) cos(th4(t))

   #5 == cos(th4(t)) sin(th1(t)) + cos(th1(t)) cos(th2(t)) sin(th4(t))

   #6 == cos(th1(t)) sin(th4(t)) + cos(th2(t)) cos(th4(t)) sin(th1(t))

   #7 == _____
         l3(t) #17 + #17 __
                          l4 - #17 #19 __
                                       l5 - #20 #18 #16 __
                                                        l5

   #8 == #14 #20 - #17 #13 #15

   #9 == #13 #15 - #14 #17 #20

   #10 == #14 #15 + #17 #20 #13

   #11 == #20 #13 + #14 #17 #15

   #12 == #17 #18 - #20 #16 #19

   #13 == sin(_______
              th1(t))

   #14 == cos(_______
              th1(t))

   #15 == sin(_______
              th4(t))

   #16 == sin(_______
              th2(t))

   #17 == cos(_______
              th2(t))

              ______
```

```
   #18 == cos(th5(t))

                 ______
   #19 == sin(th5(t))

                 ______
   #20 == cos(th4(t))
```

```
K_Total= simplify (K1+K2+K3+K3+K4+K5+K6);
disp('Energía Cinética Total');
```

```
Energía Cinética Total
```

```
pretty (K_Total);
```

```
                                                        /              _______      ___         \
Izz1 #51   Izz2 #51        /      #57   #56 \        __ |             th2p(t) #65 lc3       | / lc3 sin(th1(t)) th2p
-------- + -------- + Izz5 | #9 + --- + --- | #19 + m3 | #25 - #28 + --------------- + #37 | | ---------------------
    2          2           \       2     2  /          \                    2              / \            2

where

   #1 == l4 cos(th2(t)) + cos(th2(t)) l3(t) - l5 cos(th2(t)) sin(th5(t)) - l5 cos(th4(t)) cos(th5(t)) sin(th2(t))

   #2 == sin(th1(t)) sin(th2(t)) l3p(t)

   #3 == cos(th1(t)) sin(th2(t)) l3p(t)

   #4 == cos(th2(t)) cos(th5(t)) - cos(th4(t)) sin(th2(t)) sin(th5(t))

   #5 == sin(th1(t)) sin(th4(t)) - cos(th1(t)) cos(th2(t)) cos(th4(t))

   #6 == cos(th1(t)) sin(th4(t)) + cos(th2(t)) cos(th4(t)) sin(th1(t))

   #7 == cos(th2(t)) l3p(t)

   #8 == cos(th1(t)) cos(th2(t)) l3(t) th2p(t)

         _______
         th1p(t)
   #9 == -------
            2

   #10 == cos(th2(t)) sin(th1(t)) l3(t) th2p(t)

                       lc2 th2p(t)
   #11 == l2 th1p(t) - -----------
                            2


   #12 == _____ + __
          l3(t)   l4

   #13 == 4 l2 lc2 th1p(t) th2p(t)

   #14 == sin(th2(t)) l3(t) th2p(t)

   #15 == l4 sin(th1(t)) sin(th2(t))
```

#16 == l4 cos(th1(t)) sin(th2(t))

#17 == th5p(t) #43 + #44 - #42

#18 == th5p(t) #46 + #47 + #45

#19 == th1p(t) + #48 + #49

#20 == l2 #50 $|lc2|^2$ th1p(t) - 2 lc2 #51 $|l2|^2$ th2p(t)

#21 == th1p(t) (#53 - #52)

#22 == th1p(t) (#55 + #54)

#23 == $\overline{l3(t)}$ #67 + #67 $\overline{l4}$ - #67 #63 $\overline{l5}$ - #68 #62 #64 $\overline{l5}$

#24 == $\overline{l3p(t)}$ #66 #64

#25 == $\overline{l3p(t)}$ #65 #64

#26 == $\overline{l3p(t)}$ #67

#27 == $\overline{th1p(t)}$ + #57 + #56

#28 == $\overline{th1p(t)}$ (#65 $\overline{l2}$ - #58)

#29 == $\overline{th1p(t)}$ (#66 $\overline{l2}$ + #59)

#30 == $\overline{th4p(t)}$ #66 #64

#31 == $\overline{th5p(t)}$ #60

#32 == $\overline{th5p(t)}$ #61

#33 == $\overline{th4p(t)}$ #65 #64

#34 == $\overline{th2p(t)}$ #66

#35 == $\overline{th2p(t)}$ #65

#36 == $\overline{l3(t)}$ $\overline{th2p(t)}$ #66 #67

#37 == $\overline{l3(t)}$ $\overline{th2p(t)}$ #67 #65

#38 == $\overline{l3(t)}$ $\overline{th2p(t)}$ #64

#39 == #65 #69 - #66 #67 #68

```
#40 == #66 #69 + #67 #68 #65

#41 == #67 #62 - #68 #64 #63

#42 == cos(th1(t)) sin(th2(t)) th4p(t)

#43 == cos(th4(t)) sin(th1(t)) + cos(th1(t)) cos(th2(t)) sin(th4(t))

#44 == sin(th1(t)) th2p(t)

#45 == sin(th1(t)) sin(th2(t)) th4p(t)

#46 == cos(th1(t)) cos(th4(t)) - cos(th2(t)) sin(th1(t)) sin(th4(t))

#47 == cos(th1(t)) th2p(t)

#48 == cos(th2(t)) th4p(t)

#49 == sin(th2(t)) sin(th4(t)) th5p(t)

                2
#50 == |th2p(t)|

                2
#51 == |th1p(t)|

#52 == cos(th1(t)) sin(th2(t)) l3(t)

#53 == l2 sin(th1(t))

#54 == sin(th1(t)) sin(th2(t)) l3(t)

#55 == l2 cos(th1(t))

       _______
#56 == th5p(t) #64 #69

       _______
#57 == th4p(t) #67

       ______
#58 == l3(t) #66 #64

       ______
#59 == l3(t) #65 #64

#60 == #66 #68 - #67 #65 #69

#61 == #68 #65 + #66 #67 #69

            ______
#62 == cos(th5(t))

            ______
#63 == sin(th5(t))

            ______
#64 == sin(th2(t))

            ______
#65 == sin(th1(t))

            ______
```

```
#66 == cos(th1(t))

#67 == cos(th2(t))

#68 == cos(th4(t))

#69 == sin(th4(t))
```

```
%Energia Potencial  p=mgh

%Obtenemos las alturas respecto a la gravedad
 h1= P01(2); %Tomo la altura paralela al eje y
 h2= P12(3); %Tomo la altura paralela al eje y
 h3= P23(1); %Tomo la altura paralela al eje y
 h4= P34(1); %Tomo la altura paralela al eje y
 h5= P45(1); %Tomo la altura paralela al eje y
 h6= P56(1); %Tomo la altura paralela al eje y

 U1=m1*g*h1
```

U1 = 0

```
 U2=m2*g*h2
```

U2 =

$$\frac{g\,\mathrm{lc}_2\,m_2}{2}$$

```
 U3=m3*g*h3
```

U3 = 0

```
 U4=m4*g*h4
```

U4 = 0

```
 U5=m5*g*h5
```

U5 =

$$\frac{g\,\mathrm{lc}_5\,m_5\cos(\mathrm{th}_5(t))}{2}$$

```
 U6=m6*g*h6
```

U6 = 0

```
 %Calculamos la energía potencial total
 U_Total= U1 + U2 + U3 + U4 +U5 +U6;
```

```
 %Obtenemos el Lagrangiano
 Lagrangiano= simplify (K_Total-U_Total);
 %pretty (Lagrangiano);

%Modelo de Energía
 H= simplify (K_Total+U_Total);
  %pretty (H)
```